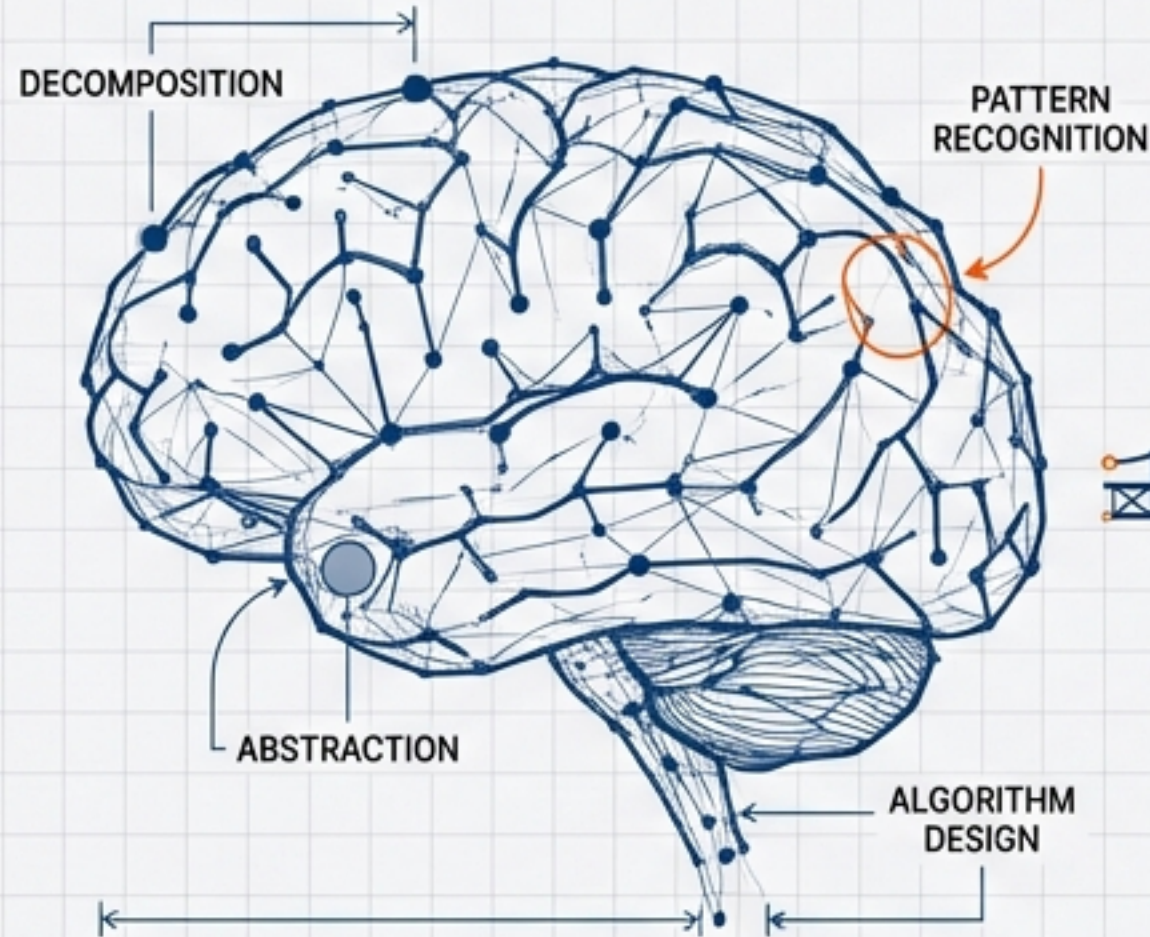


From Abstract Thought to Concrete Structure

Mastering Computational Thinking to Build Robust Data Structures

The Mindset (Abstract)



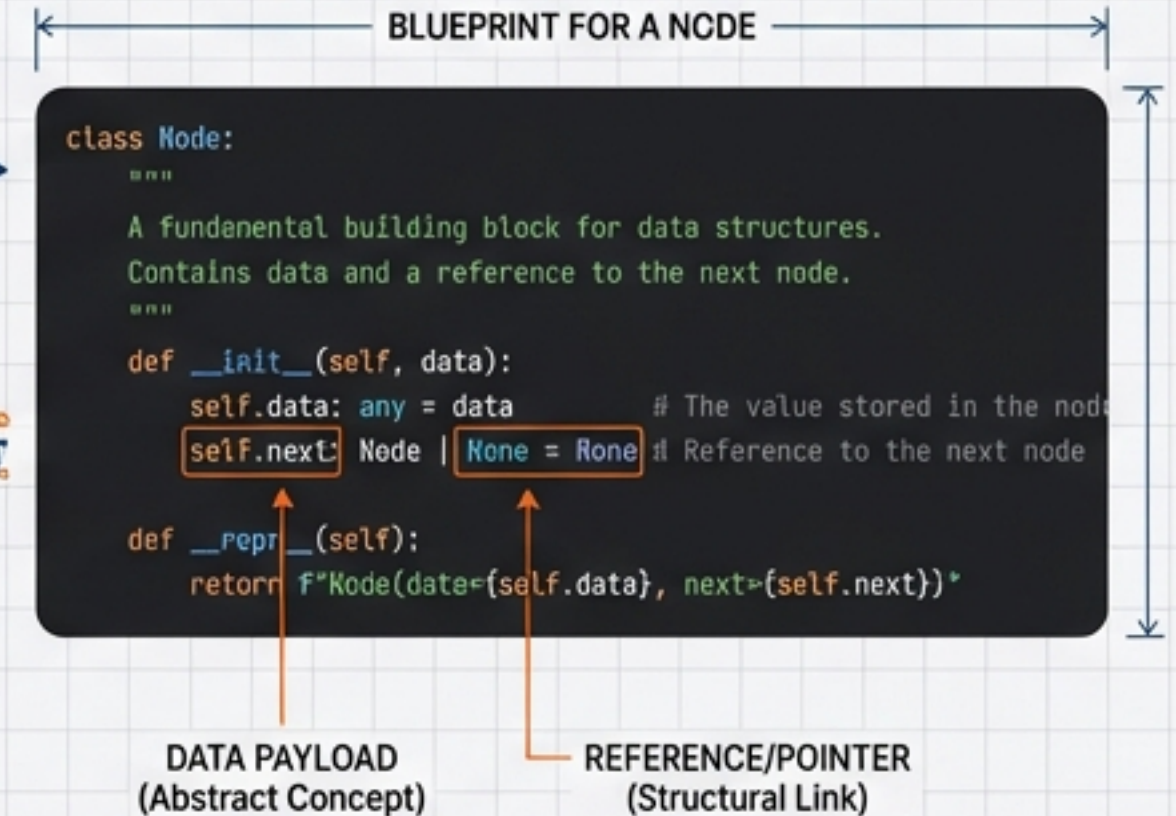
- Conceptualize complex problems into manageable components.
- Identify recurring patterns and relationships.
- Focus on essential characteristics, ignoring non-essentials.
- Formulate a step-by-step process to solve the problem.

Code is just the translation of logic.

The Engineer's Mindset



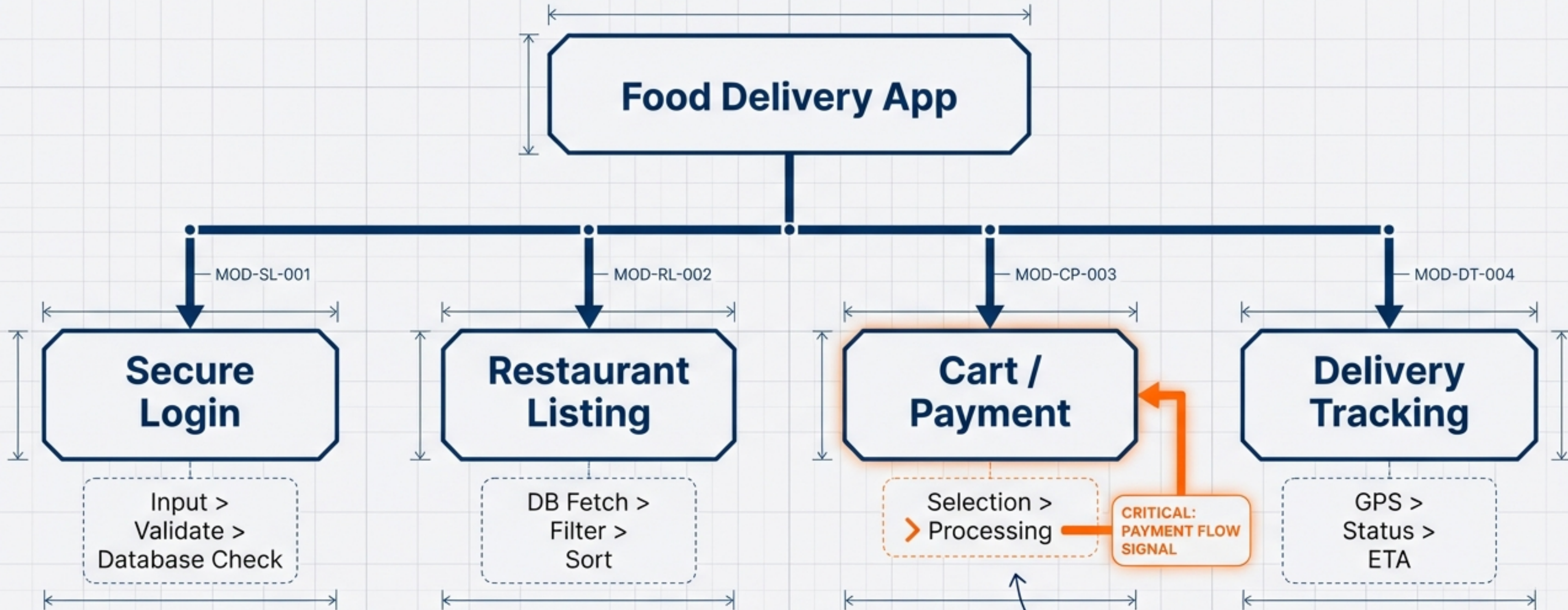
The Structure (Concrete)



- Implements abstract concepts into tangible code.
- Defines the building block for linked lists, trees, and graphs.
- Establishes clear invariants and relationships.
- Enables efficient data manipulation and storage.

An engineer does not start by writing code. They start by decomposing the problem, recognizing patterns, and defining invariants.

Pillar 1: Decomposition – Breaking the Monolith

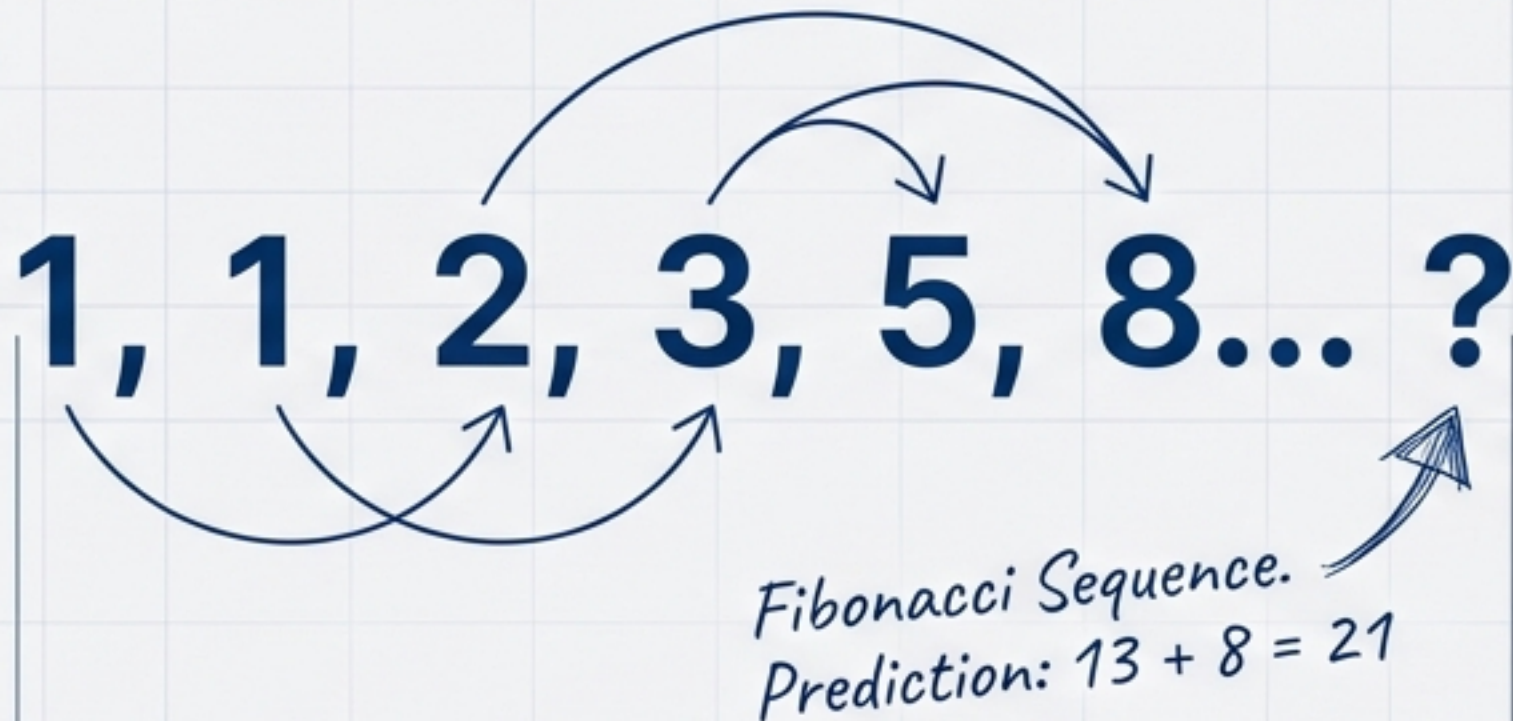


We cannot build 'The App.' We build the components that comprise the app. Complex problems are just a collection of smaller, solvable problems.

Meta Thinking: This structure is modular. We can iterate on "Restaurant Listing" without breaking "Secure Login". The sum of parts = The Whole. -ENG. LEAD

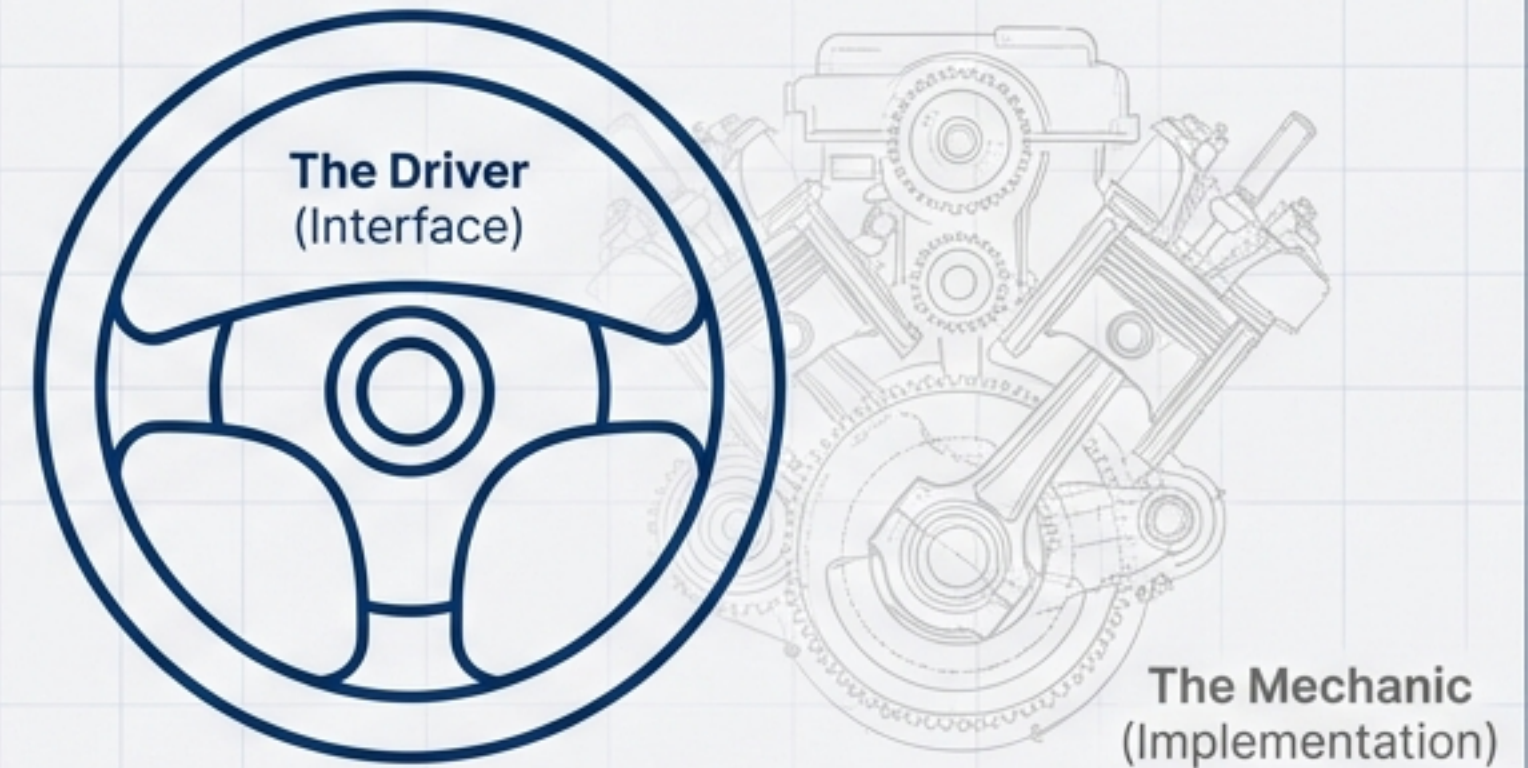
Pillars 2 & 3: Pattern Recognition & Abstraction

Pattern Recognition



Identify sequences to predict future data without recalculating.

Abstraction



```
max(array)
```

We don't need to see the loop to find the largest number.



SCALE: 1 UNIT = 1 MODULE

Meta Thinking: Filtering Signal from Noise

Query: When will he turn 25?

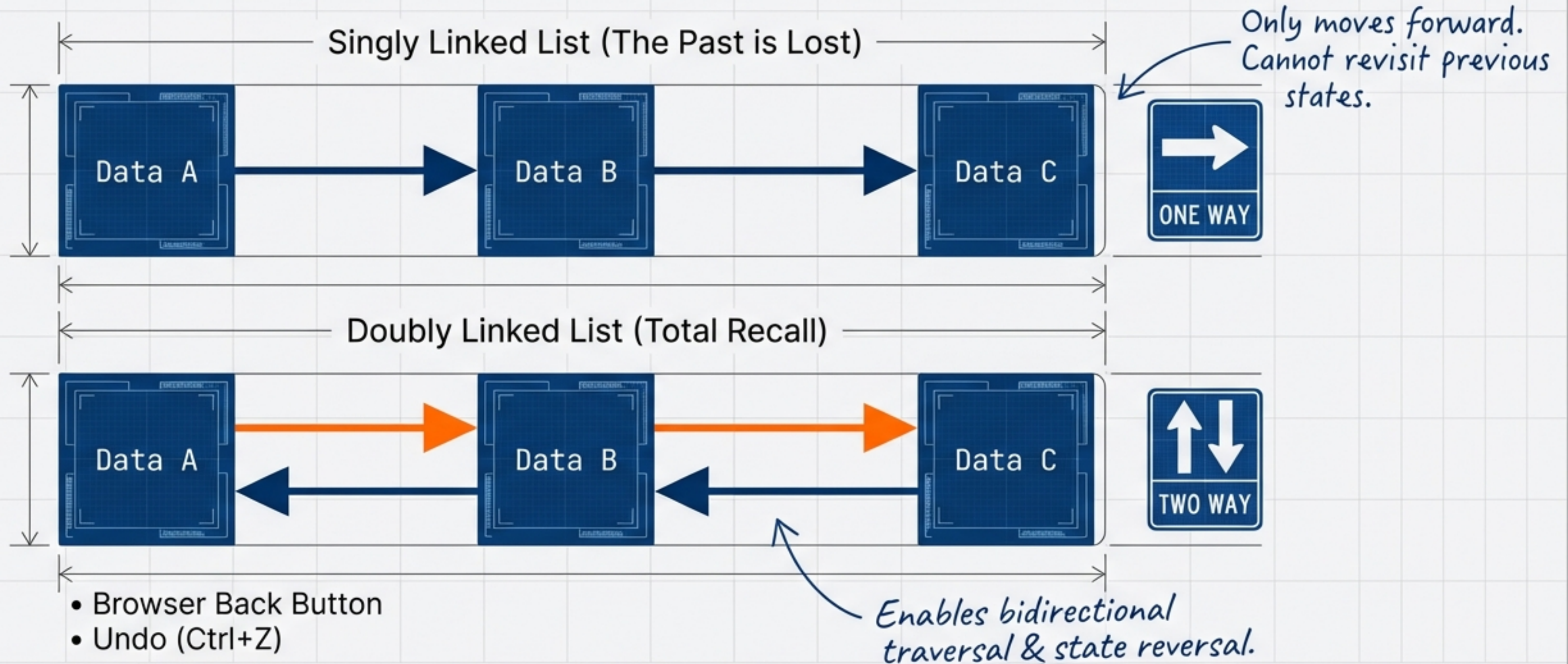


Algorithm: Target (25) - Current (20) = 5 Years to Add.

→ Always ask: What is the minimal information needed to solve this specific query?



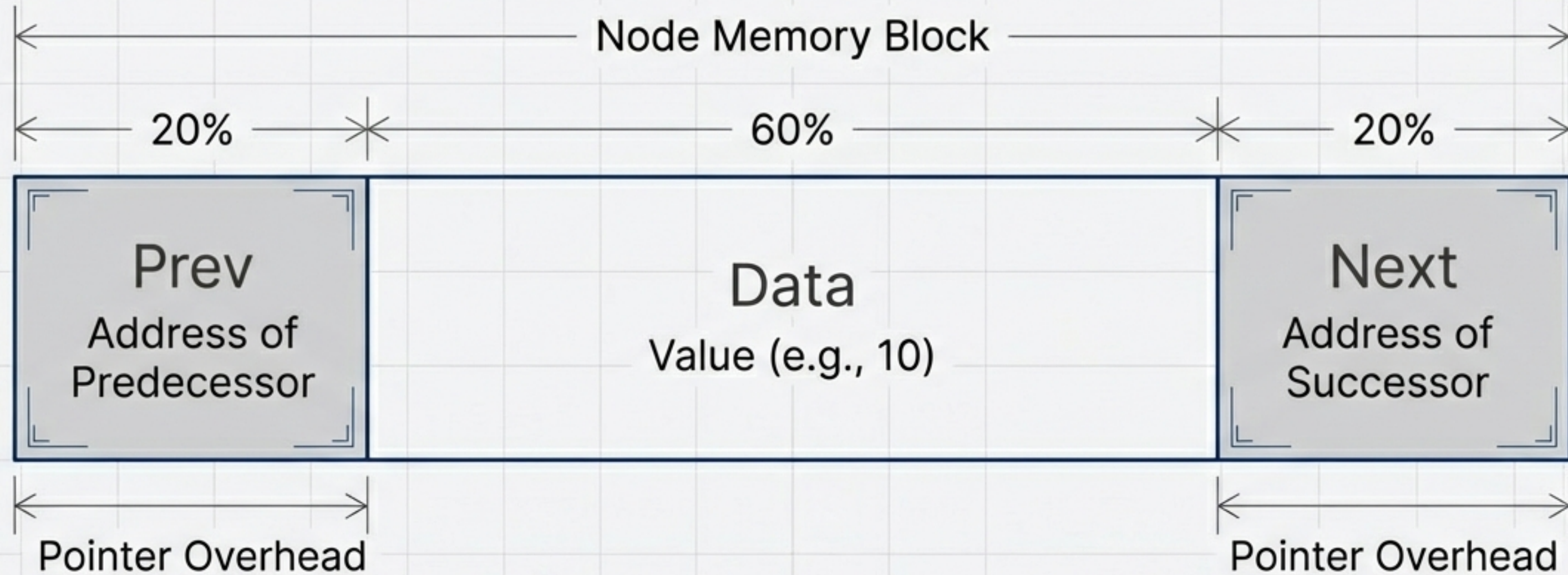
The Structural Solution: The Doubly Linked List



Singly lists have a blind spot: the past.
Doubly linked lists remember where they came from.



Anatomy of a Node



We trade memory for mobility.

```
class Node:
    def __init__(self, value):
        self.data = value
        self.next = None
        self.prev = None
```

Mechanics: Insertion at the Beginning

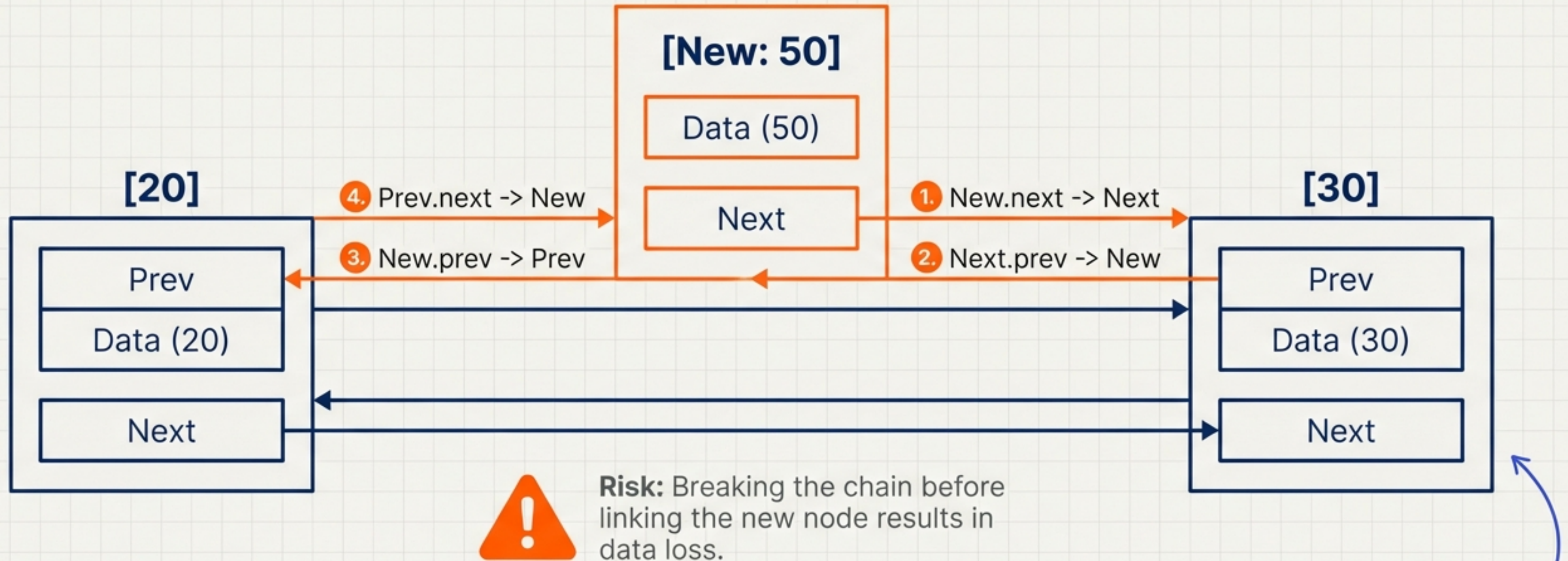


The **Invariant Check**: If Head is not None, we must update the old head's 'prev' pointer. Failing to do so breaks the backward chain.

```
if self.head is not None:  
    self.head.prev = new_node  
self.head = new_node
```

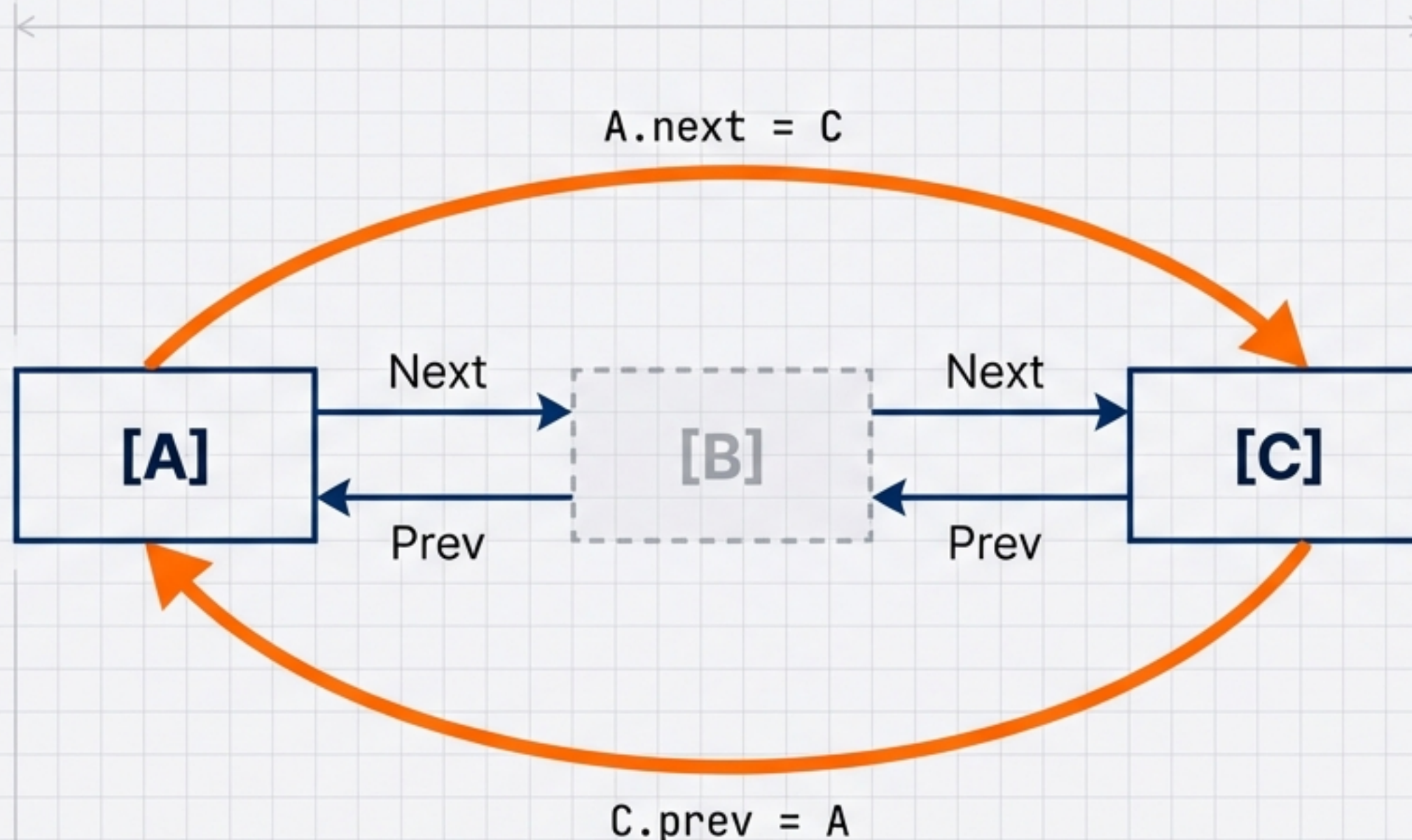
Dark Mode

Surgical Precision: Insertion in the Middle



Search first (Traversal), then perform surgery (Insertion).

Deletion: Building the Bridge



1. Find Node 't'.
2. Link Predecessor to Successor.
3. Link Successor to Predecessor.

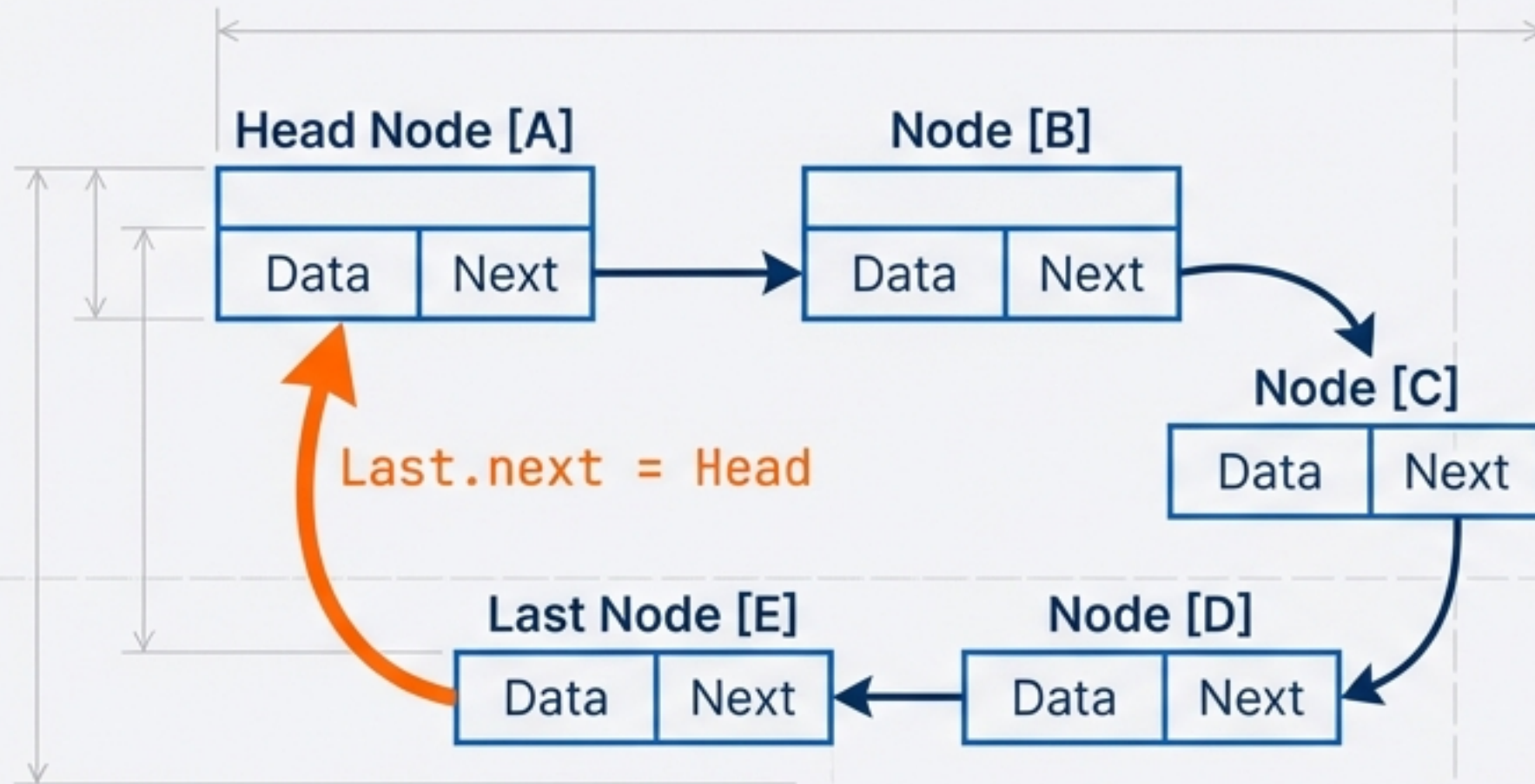
By building a bridge over Node B, we remove it from the sequence. The Garbage Collector cleans up the rest.



Edge Case Alert

Handle Head and Tail deletions explicitly (Next is None / Prev is None).

Closing the Loop: The Circular Linked List



Linear:

Ends at None

Circular:

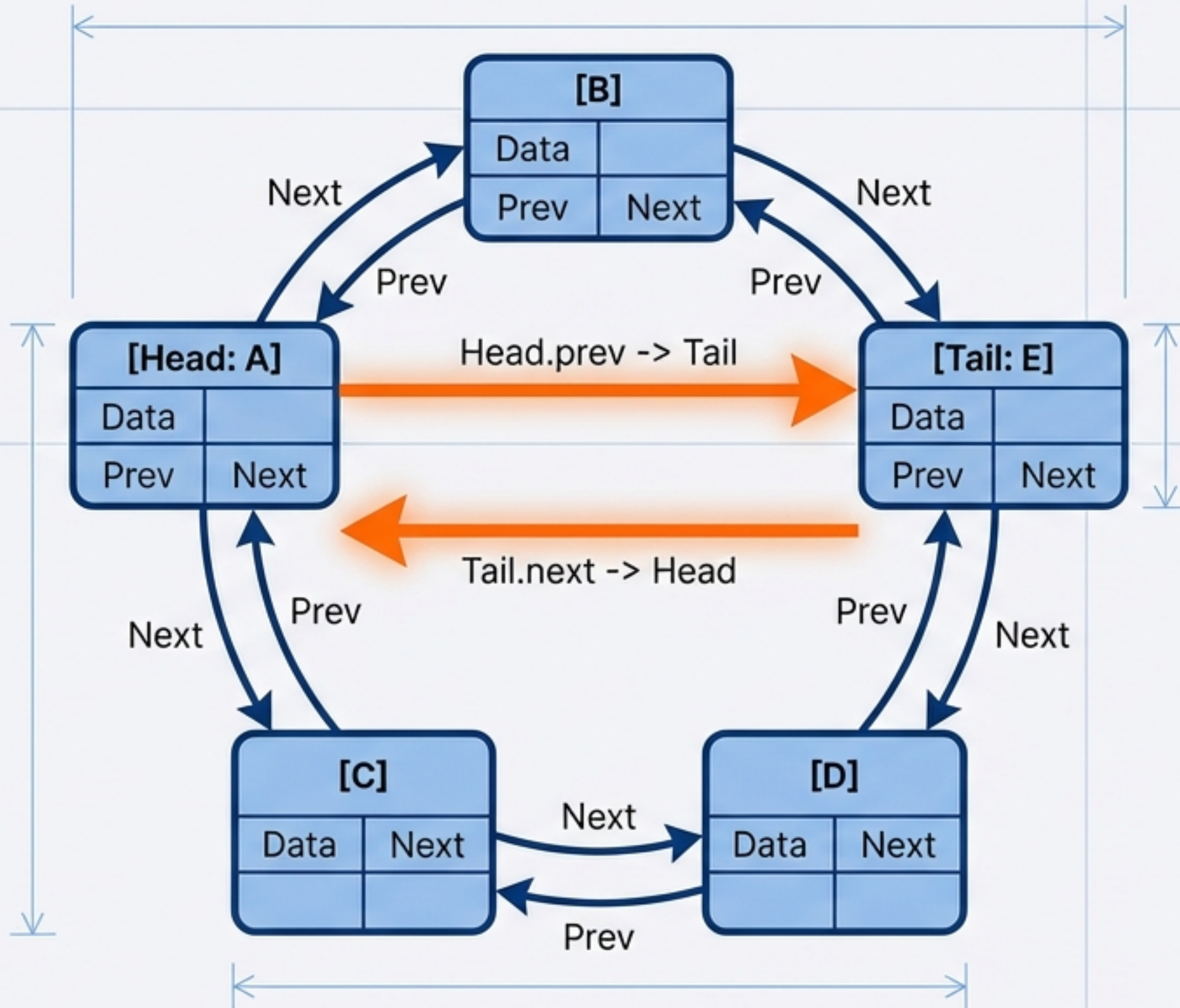
Infinite Loop

Linear: ~~while t.next is not None:~~

Circular: **while t.next != self.head:**

*The Invariant changes:
Traversal stops only when
we return to the start.*

The Ultimate Structure: Circular Doubly Linked List



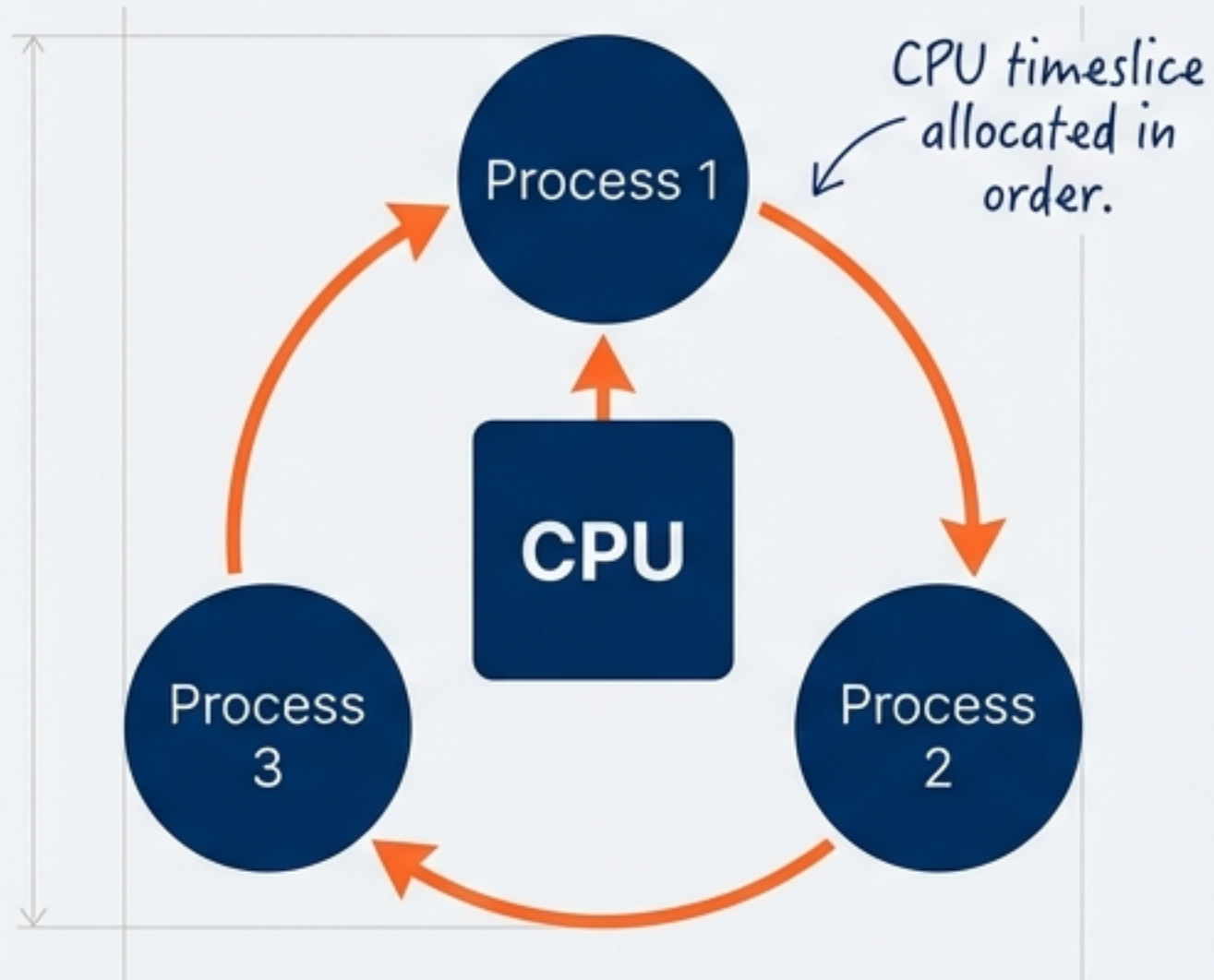
Why do this? From Head, you can immediately access the Tail (Head.prev) without traversing the list.

O(1) Access to End.

```
last_node.next = self.head  
self.head.prev = last_node
```

Where Theory Meets Reality

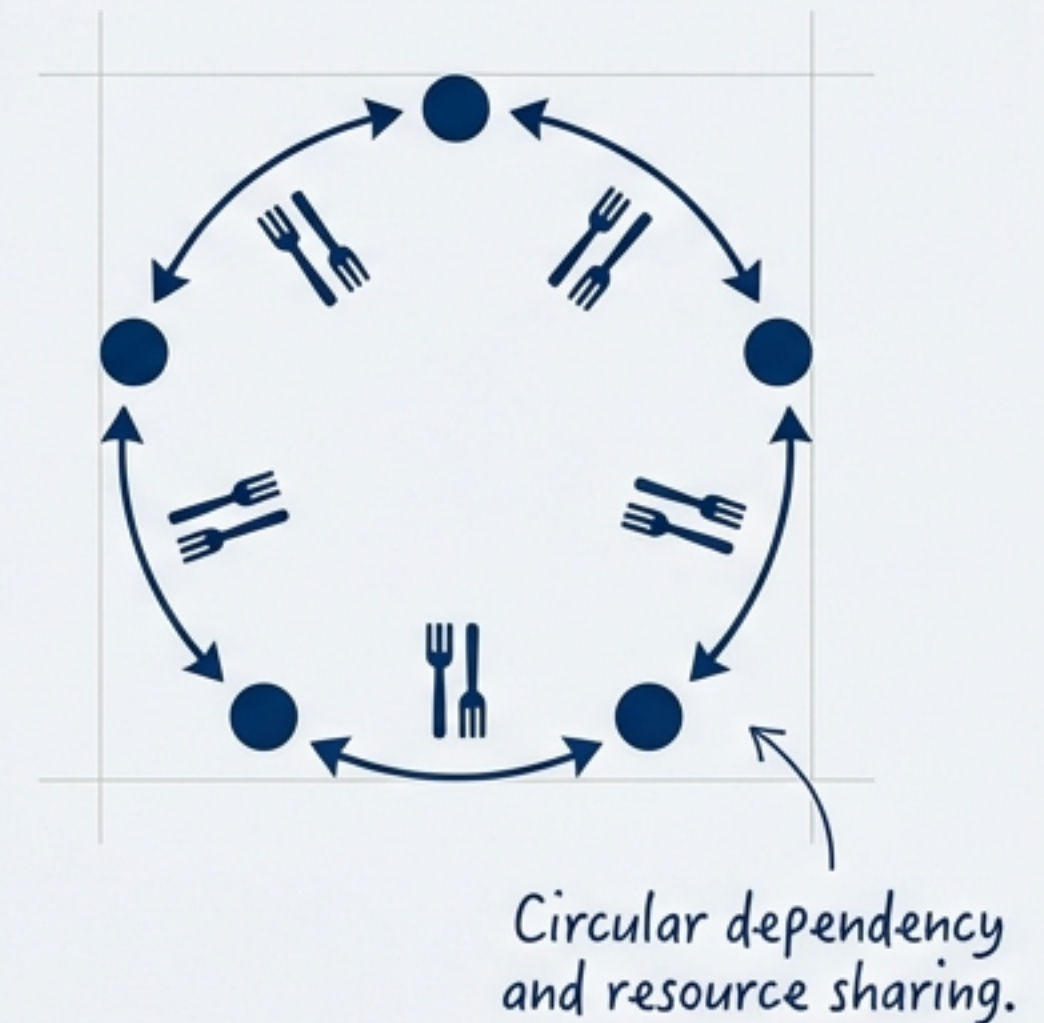
Round Robin Scheduling (OS)



Media Playlists



Dining Philosophers

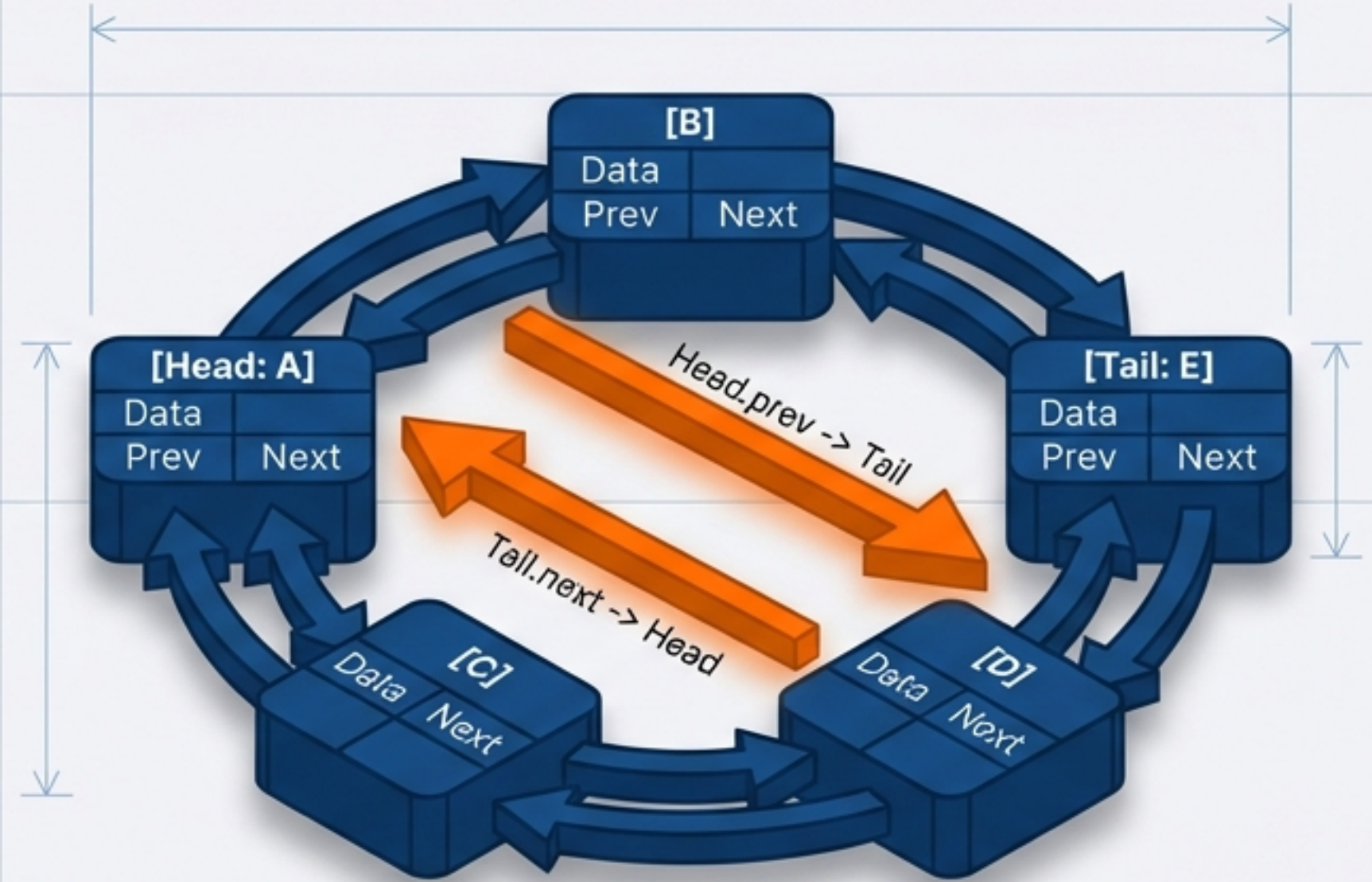


We use Circular Linked Lists when the process has no true end.

The Engineer's Artifact

- **Key Takeaways**

1. **Think First:** Decompose the problem.
2. **Find Invariants:** Identify the truths that never change.
3. **Handle Edge Cases:** Code fails at the edges (Head, Tail, None).



**“Syntax is temporary. Logic is permanent.
Master the thinking, and the code will follow.”**